🎓 MCA Final Year Viva Voce Preparation Guide: AI Resume & Job Match Analyzer

---

📌 1. Project Overview & Objective

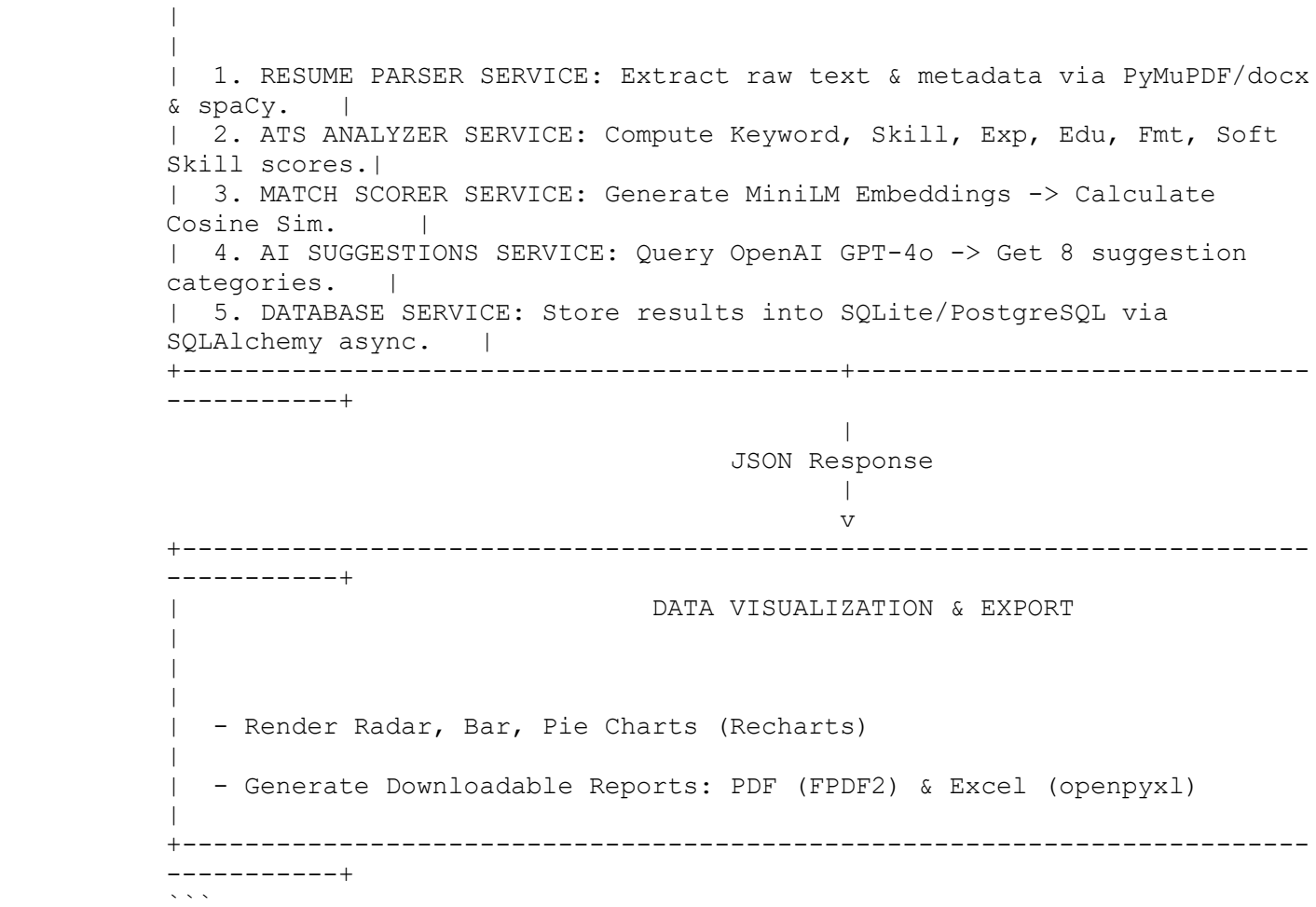
Project Title: AI Resume & Job Match Analyzer
Domain: Data Analytics, Natural Language Processing (NLP), Machine Learning (ML), Full-Stack Web Development

Problem Statement:
Traditional job applications often fail to pass Applicant Tracking Systems (ATS) because job seekers do not know how ATS software evaluates resumes against job descriptions. Candidates lack insight into keyword alignment, formatting compliance, missing technical skills, and semantic relevance.

Solution Offered:
An end-to-end, production-ready web application that:
1. **Parses Resumes** (PDF & DOCX) into structured JSON data using PyMuPDF, `pdfplumber`, and `python-docx`.
2. **Calculates ATS Scores** using multi-vector weighted scoring algorithms.
3. **Calculates Semantic Match Scores** using Deep Learning (Sentence Transformers `all-MiniLM-L6-v2` dense vector embeddings & Cosine Similarity) alongside TF-IDF fallbacks.
4. **Generates Actionable AI Suggestions** using OpenAI GPT-4o (structured JSON mode) and rule-based fallback engines.
5. **Researches Target Companies** providing source-cited analytics (Glassdoor, LinkedIn, AmbitionBox, Indeed).
6. **Visualizes Analytics** via an interactive React + Recharts dashboard and exports PDF/Excel reports.

📐 2. System Architecture & End-to-End Data Flow

```
+-----------------------------------------------------------------------------+
|                                  FRONTEND (React 18 + Vite)
|
|
|
|  [ User Uploads Resume ] ---> [ Inputs Job Description ] ---> [ Clicks Analyze ]  |
+-----------------------------------------------+-----------------------------+
                                                |
                                      HTTP POST /api/analysis/run
                                                |
                                                v
+-----------------------------------------------------------------------------+
|                                   BACKEND (FastAPI)
|
```



🛠️ 3. Tech Stack Justification (For Viva)

Layer	Technology Used	Viva Justification / Why chosen?
Frontend Framework	React 18 + Vite	Component-based UI, fast hot module replacement (HMR), lightweight DOM rendering.
Styling	Vanilla CSS + Tailwind v4	Glassmorphic design system, responsive UI, micro-animations.
Visualization	Recharts	Declarative SVG-based charting library for React (Radar, Bar, Pie charts).
Backend Framework	Python 3.11 + FastAPI	Asynchronous I/O (`async`/`await`), auto OpenAPI documentation (`/api/docs`), type validation via Pydantic v2.
ORM & Database	SQLAlchemy (Async) + SQLite	Async database driver (`aiosqlite`) supporting non-blocking DB queries; easily scalable to PostgreSQL in production.
PDF/DOCX Extraction	PyMuPDF (`fitz`), `pdfplumber`, `python-docx`	Dual-layer fallback strategy: PyMuPDF for speed, `pdfplumber` for complex layouts/tables, `python-docx` for MS Word files.
NLP & NER	spaCy (`en_core_web_sm`) + Regex	Named Entity Recognition (NER) for extracting candidate names, emails, phones, and technical skills.

Neural Embeddings	Sentence Transformers (`all-MiniLM-L6-v2`)	Maps text to 384-dimensional dense vectors to capture semantic meaning (contextual similarity beyond exact keyword matches).
Classical ML	scikit-learn (`TfidfVectorizer` + Cosine Similarity)	Lightweight mathematical fallback if neural transformer model is unavailable.
Generative AI	OpenAI GPT-4o (`AsyncOpenAI`)	Provides natural language resume rewrites, project recommendations, and certification suggestions using structured JSON mode.
Export Engines	FPDF2 & OpenPyXL	Programmatically generates formatted PDF summary cards and multi-tab Excel workbooks.

🧮 4. Detailed Working of Core Modules & Algorithms

Module 1: Resume Parsing (`backend/services/resume_parser.py` & `backend/utils/nlp_utils.py`)
1. **Text Extraction Pipeline:**
   - Evaluates file extension (`.pdf` or `.docx`).
   - For PDFs: Tries `fitz.open()` (PyMuPDF). If text length < 50 characters, falls back to `pdfplumber.open()`.
   - For DOCX: Iterates through paragraphs and table cells using `python-docx`.
2. **Entity & Skill Extraction:**
   - **Name:** Heuristic parser checking top 5 lines, ignoring blacklisted headers like `"Curriculum Vitae"`, `"Resume"`, `"Contact"`.
   - **Email & Phone:** Extracted using regular expressions:
     - Email: `r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'`
     - Phone: Matches 10-digit formats and international country code formats `+91`.
   - **Skills:** Regex with word-boundary lookarounds `(?<![a-zA-Z0-9])skill(?![a-zA-Z0-9])` against a dictionary of tech skills (Python, SQL, PyTorch, React, etc.) and soft skills (Leadership, Communication, Agile).

Module 2: ATS Scoring Engine (`backend/services/ats_analyzer.py`)

The overall ATS score ($0 - 100$) is computed using a weighted multi-vector equation:

$$\text{ATS Score} = (S_{\text{kw}} \times 0.30) + (S_{\text{skill}} \times 0.25) + (S_{\text{exp}} \times 0.20) + (S_{\text{edu}} \times 0.10) + (S_{\text{fmt}} \times 0.10) + (S_{\text{soft}} \times 0.05)$$

Breakdown of Component Scores:
1. **Keyword Score (S_{kw} - 30% weight):**
   - Extracts top 60 keywords from Job Description (JD) and top 100 from Resume using TF-IDF.
   - $S_{\text{kw}} = \frac{|\text{Matched Keywords}|}{|\text{Total JD Keywords}|} \times 100$
2. **Technical Skill Match (S_{skill} - 25% weight):**
   - Calculates set intersection: $\text{Matched Skills} = \text{JD Skills} \cap \text{Resume Skills}$.
   - $S_{\text{skill}} = \frac{|\text{Matched Skills}|}{|\text{Total JD Skills}|} \times 100$

3. **Experience Match (S_{exp} - 20% weight):**
   - Parses experience requirements from JD using regex (e.g., `"3 to 5 years"` $\rightarrow \text{min}=3$).
   - Compares candidate's extracted experience years against JD requirements. Returns 100% if requirement met, or scaled down ratio if lower.
4. **Education Match (S_{edu} - 10% weight):**
   - Degree level hierarchy: $\text{PhD/Doctorate} = 4, \text{Master/MCA/M.Tech} = 3, \text{Bachelor/BCA/B.Tech} = 2, \text{Diploma} = 1$.
   - If Candidate Level $\ge$ JD Level $\rightarrow 100\%$; if 1 tier lower $\rightarrow 70\%$; else $\rightarrow 40\%$.
5. **Formatting Score (S_{fmt} - 10% weight):**
   - Heuristic formatting check that deducts points for excessive all-caps words, text length <200 words or >2000 words, or missing standard sections (`Experience`, `Education`, `Skills`).
6. **Soft Skill Match (S_{soft} - 5% weight):**
   - Percentage overlap of soft skills mentioned in JD versus resume.

Module 3: Match Scorer & Semantic Embeddings (`backend/services/match_scorer.py`)

Whereas ATS scoring checks explicit keyword matches, the **Match Scorer** measures contextual semantic similarity.

Algorithm Flow:
1. **Vector Embedding Generation:**
   - Uses `SentenceTransformer("all-MiniLM-L6-v2")` to encode the first 3000 characters of the resume text and job description into 384-dimensional dense vectors $\vec{u}$ and $\vec{v}$.
2. **Cosine Similarity Computation:**
   $$\text{Semantic Similarity} = \cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|} = \frac{\sum_{i=1}^{n} u_i v_i}{\sqrt{\sum_{i=1}^{n} u_i^2} \sqrt{\sum_{i=1}^{n} v_i^2}}$$
3. **TF-IDF Vector Space Fallback:**
   - If Sentence Transformers package is missing or fails, it converts texts into TF-IDF term vectors using `scikit-learn` and calculates matrix cosine similarity.
4. **Final Match Score Formula:**
   $$\text{Final Match Score} = (\text{Semantic Similarity} \times 0.40) + (\text{ATS Weighted Score} \times 0.60)$$
5. **Match Category Classification:**
   - $\ge 80\% \rightarrow \mathbf{Very\ High}$
   - $60\% - 79\% \rightarrow \mathbf{High}$
   - $40\% - 59\% \rightarrow \mathbf{Medium}$
   - $< 40\% \rightarrow \mathbf{Low}$

Module 4: AI Suggestions Service (`backend/services/ai_suggestions.py`)

1. **GPT-4o Integration:**
   - Constructs a prompt containing resume snippet, JD snippet, ATS score, matched skills, and missing skills.

   - Enforces structured JSON output via `response_format={"type": "json_object"}`.
   - Returns 8 suggestion categories:
     1. Suggested Skills
     2. Recommended Certifications
     3. Project Ideas
     4. Resume Bullet Rewrites
     5. High-Impact Action Verbs
     6. Grammar Improvements
     7. Target Keywords to Add
     8. Metric Quantification Tips (e.g. adding % improvements)
2. **Fault Tolerance:** Uses `tenacity` library to retry failed API calls up to 3 times with exponential backoff ($2s \rightarrow 4s \rightarrow 8s$). If API key is missing or quota exceeded, seamlessly falls back to a local rule-based suggestion engine.

Module 5: Company Analyzer Service (`backend/services/company_analyzer.py`)

Allows candidates to research company environment, salaries, and culture.
- Generates 6 rating dimensions ($0.0 - 10.0$): Overall Rating, Work-Life Balance, Salary Satisfaction, Career Growth, Culture Rating, Interview Difficulty.
- Provides bullet points for Pros, Cons, and Overall Recommendation.
- **Source Citations:** Generates active source links to Glassdoor, AmbitionBox, LinkedIn, Indeed, and Google Search for verification.

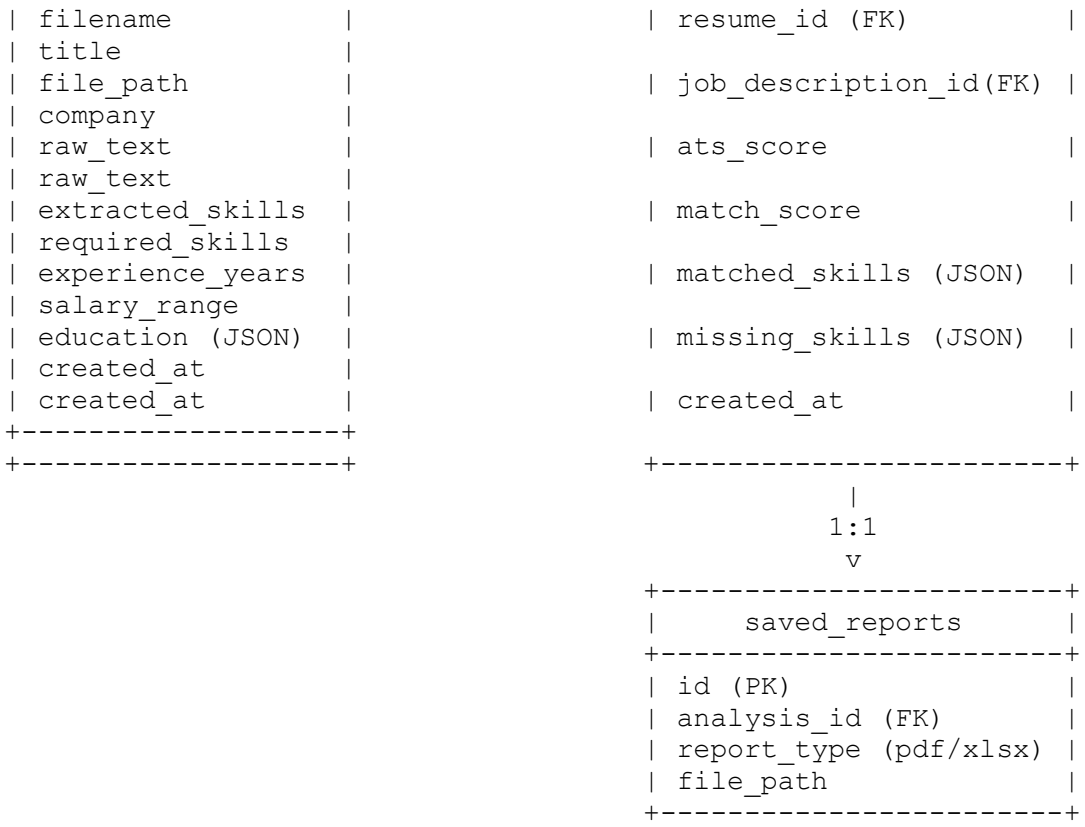
Module 6: Report Exporter Service (`backend/services/report_exporter.py`)

- **PDF Generation (`FPDF2`):** Generates a multi-page PDF report complete with color-coded score badges, key missing skills, and AI suggestions.
- **Excel Generation (`openpyxl`):** Generates a multi-sheet spreadsheet:
  - Sheet 1: Executive Summary & Scores.
  - Sheet 2: Matched & Missing Keywords/Skills.
  - Sheet 3: Actionable Recommendations.

🗄️ 5. Database Schema & Models (`backend/models/models.py`)

The application uses **SQLAlchemy ORM** connected asynchronously via `aiosqlite` to SQLite (database file: `ats_analyzer.db`).

```
+--------------------+       1:N       +-------------------------+
1:N       +--------------------+
|      resumes       | <-------------- |    resume_analyses      | --------
------> |  job_descriptions |
+--------------------+                 +-------------------------+
+--------------------+
| id (PK)            |                 | id (PK)                 |
| id (PK)            |
```



💻 6. Frontend Pages & UI Workflow

Page Route	React Component	Functionality
`/`	`HomePage.jsx`	Landing page explaining ATS concepts, features, workflow steps, and call-to-actions.
`/resume-analyzer`	`ResumeAnalyzerPage.jsx`	File dropzone (PDF/DOCX upload), Job description textarea, Real-time progress bar, Detailed Analysis Dashboard.
`/company-analyzer`	`CompanyAnalyzerPage.jsx`	Search input for company name/URL, 6 rating cards, Pros/Cons lists, External source citations.
`/dashboard`	`DashboardPage.jsx`	History table of all previous resume analyses stored in DB, allowing re-inspection and report downloads.
`/about`	`AboutPage.jsx`	Academic project overview, tech stack badges, system workflow diagrams.
`/docs`	`DocumentationPage.jsx`	User manual explaining ATS scoring breakdown, tips to increase score, and FAQ.

❓ 7. Top MCA Viva Questions & Sample Answers

Q1: What is an ATS and how does your project simulate it?
Answer: An Applicant Tracking System (ATS) is software used by recruiters to filter job applications based on keyword matching, formatting, experience, and skills. Our project simulates an ATS by

parsing resumes into structured text using PyMuPDF/python-docx, extracting entities using spaCy and Regex, and scoring candidates against job descriptions across 6 weighted categories (Keywords, Skills, Experience, Education, Formatting, Soft Skills).

Q2: What is the difference between Keyword Matching and Semantic Matching in your system?
Answer:
- **Keyword Matching** is rule-based lexical matching. It checks if exact strings (e.g., `"Python"`, `"Machine Learning"`) exist in both the resume and JD.
- **Semantic Matching** uses Deep Learning embeddings (`SentenceTransformer - all-MiniLM-L6-v2`). It converts sentences into 384-dimensional dense vectors and calculates Cosine Similarity. This allows the system to recognize that *"built neural networks"* is semantically similar to *"deep learning experience"*, even if exact words differ.

Q3: Explain Cosine Similarity and its formula.
Answer: Cosine similarity measures the cosine of the angle between two multi-dimensional vectors in a vector space. It evaluates how similar two documents are regardless of their length.
$$\text{Cosine Similarity} = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$$
- Value ranges from -1 (opposite) to 1 (identical). In our text similarity context, it ranges from 0.0 to 1.0 (scaled to $0 - 100\%$).

Q4: How do you handle cases where OpenAI API is down or no API key is provided?
Answer: We implemented a **Graceful Fallback Mechanism**:
1. For suggestions: If OpenAI GPT-4o fails or is unconfigured, the system automatically triggers `_rule_based_suggestions()`, which analyzes missing skills/keywords locally and returns structured recommendations.
2. For embeddings: If `SentenceTransformer` is unavailable, `_tfidf_similarity()` calculates cosine similarity using `scikit-learn`'s `TfidfVectorizer`.
3. For parsing: If `PyMuPDF` fails on a PDF, `pdfplumber` acts as the secondary parser.

Q5: Why did you choose FastAPI over Flask or Django?
Answer:
- **Asynchronous Native (`async`/`await`):** Handles concurrent heavy non-blocking I/O operations (file reads, DB queries, external API calls) efficiently.
- **High Performance:** Built on Starlette and Pydantic, making it one of the fastest Python web frameworks.
- **Automatic OpenAPI Documentation:** Automatically generates interactive Swagger UI (`/api/docs`) for testing endpoints.

- **Data Validation:** Pydantic schema validation prevents bad data payloads at runtime.

Q6: How are files uploaded and parsed securely?
Answer:
- Files are uploaded via standard `multipart/form-data` requests to `/api/resume/upload`.
- The server validates file extensions (`.pdf`, `.docx`) and file size before saving to an `uploads/` directory with sanitized unique filenames.
- Text extraction happens in-memory or from temporary disk reads without executing any arbitrary file content.

Q7: What database is used and how is it queried asynchronously?
Answer: We use **SQLite** managed through **SQLAlchemy Async ORM** with `aiosqlite`. Async sessions (`AsyncSession`) are created via a generator dependency (`get_db`), ensuring database connection pools are efficiently opened and closed per request without blocking the FastAPI event loop.

📌 8. Summary Checklist for Viva Presentation
- [x] Know how to start backend (`python -m uvicorn backend.main:app --reload`).
- [x] Know how to start frontend (`npm run dev` in `frontend/` directory).
- [x] Be ready to demonstrate uploading a PDF resume and pasting a sample Job Description.
- [x] Be ready to point out the Radar Chart, ATS Score breakdown, and missing skills.
- [x] Be ready to explain the weight distribution (30% Keyword, 25% Skill, 20% Exp, 10% Edu, 10% Formatting, 5% Soft Skill).
- [x] Be ready to explain Cosine Similarity and Sentence Transformer embeddings.